

15/10/23

Task-12

simulate gaming concepts using pygame

AIM:

Create basic Pygame application that simulates simple gaming concepts, including initializing a game window, displaying and moving a player object, adding an enemy object, detecting collisions, and controlling the game flow.

ALGORITHM:

1-Initialize Pygame and setup the game window:

- Import the necessary pygame modules.

- Initialize Pygame

- Set the dimensions of game window

- Create a game window with a title.

2-Set the background color and display it:

- Define the background color.

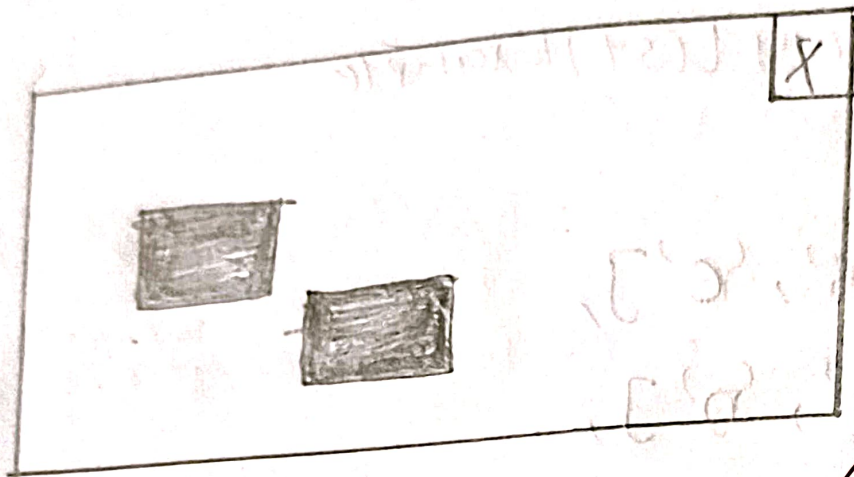
- Fill the game window with the background color

3-Create Player object:

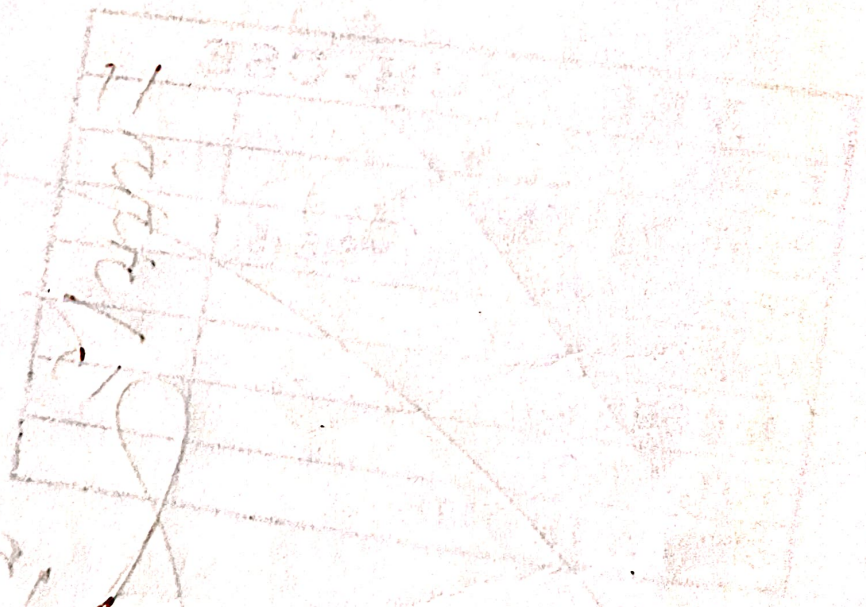
- Define the player object's properties.

(eg: shape, color, initial position)

- Display the player object on the screen.



Handwritten notes in Hindi, including the word 'प्रमाण' (Proof) and mathematical expressions like $\frac{a}{b} = \frac{c}{d}$.



Handwritten notes in Hindi, including the word 'प्रमाण' (Proof) and mathematical expressions like $\frac{a}{b} = \frac{c}{d}$.



Handwritten notes in Hindi, including the word 'प्रमाण' (Proof) and mathematical expressions like $\frac{a}{b} = \frac{c}{d}$.

4. Game loop: The game loop continuously

- Handle KEYDOWN events for arrow keys.
- Update the player object's position based on the key passed.

5. ~~Collision detection~~: ^{create an enemy object that} It moves automatically

- Define the enemy object's properties -

(eg., shape, colour, initial position, speed)

- Update the enemy object's position in each game loop iteration.

6. Detect collision between the player and enemy objects:

- Check if the player and enemy objects overlap.

• If they collide, display "Game Over" and stop the game

7. Control the game loop and frame rate:

- Use a game loop to continuously update and render the game state.

- Control the frame rate to achieve a smooth gaming experience

Program:-

```
import pygame
```

```
import sys
```

```
# Initialize Pygame
```

```
pygame.init()
```

```
# Set up game constants
```

```
WIDTH, HEIGHT = 800, 60
```

```
PLAYER_SIZE = 50
```

```
ENEMY_SIZE = 50
```

PLAYER - SPEED = 5

ENEMY - SPEED = 3

BACK GROUND - COLOR = (0,0,0) # Black

PLAYER - COLOR = (0, 255, 0) # Green

ENEMY - COLOR = (255, 0, 0) # Red

Set up the display

screen = pygame.display.set_mode(WIDTH, HEIGHT)

pygame.display.set_caption("Simple game simulation")

Set up the fonts

font = pygame.font.Font(None, 36)

Set up the player and enemy objects

player_pos = [WIDTH/2, HEIGHT/2]

enemy_pos = [0, HEIGHT/2]

Game loop

clock = pygame.time.Clock()

while True:

for event in pygame.event.get():

if event.type == pygame.QUIT:

pygame.quit()

sys.exit()

keys = pygame.key.get_pressed()

if keys[pygame.K_LEFT]:

player_pos[0] -= PLAYER_SPEED

if keys[pygame.K_RIGHT]:

player_pos[0] += PLAYER_SPEED

if keys[pygame.K_UP]:

player_pos[1] -= PLAYER_SPEED

if keys[pygame.K_DOWN]:

player_pos[1] += PLAYER_SPEED

#move the enemy

enemy_pos[0] += ENEMY_SPEED

if enemy_pos[0] > WIDTH or enemy_pos[0] < 0:

ENEMY_SPEED = -ENEMY_SPEED

#collision detection

if (player_pos[0] < enemy_pos[0] + ENEMY_SIZE and

player_pos[0] + PLAYER_SIZE > enemy_pos[0] and

player_pos[1] < enemy_pos[1] + ENEMY_SIZE and

player_pos[1] + PLAYER_SIZE > enemy_pos[1]):

screen.fill(BACKGROUND_COLOR)

text = font.render('Game over', True, (255, 255, 255))

screen.blit(text, (WIDTH/2-50, HEIGHT/2-18))

pygame.display.flip()

pygame.time.wait(2000)

break

```

# Draw every thing
screen . fill (BACKGROUND - COLOR)
pygame . draw . rect (screen, PLAYER - COLOR, (player_pos[0],
player_pos[1], PLAYER - SIZE, PLAYER - SIZE))
pygame . draw . rect (screen, ENEMY - COLOR, (enemy_pos[0],
enemy_pos[1], ENEMY - SIZE, ENEMY - SIZE))
pygame . display . flip ()

# cap the frame rate
clock . tick (60)

```

VEL TECH - CSE	
EX NO.	
PERFORMANCE (5)	12
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	32
SIGN WITH DATE	

15/10/25

RESULT:

The result is a simple demonstration of basic gaming concepts using R

15/10/25